

Backstage 三大模組介紹: Catalog、Template、TechDocs

Backstage 的三個核心模組,各解決開發者組織裡不同階段的痛點。從「**有沒有**」、「**怎麼蓋**」、「**怎麼用**」三個角度切:

	Software Catalog	Software Template	TechDocs
核心問題	「我們公司有什麼軟體?」	「我要新建一個服務,怎麼開始?」	「這個服務的文件在哪?」
狀態類型	註冊表(registry)	生成器(scaffolder)	發佈系統(publisher)
生命週期階段	服務存在後	服務出生時	服務存活中
資料來源	catalog-info.yaml (各 repo 內)	template.yaml (集中或散落都行)	mkdocs.yaml + docs/*.md (各 repo 內)
使用者	所有人(查找、追蹤)	開發者(自助開新專案)	所有人(讀文件)
類比	LinkedIn(找人/找服務)	Yeoman / create-react-app	GitBook / Confluence, 只是放 repo 裡

1. Software Catalog — 「組織裡所有軟體的中央註冊表」

解決的問題

- 「我們公司有幾個 microservice?分別誰負責?」
- 「我這個服務 down 了,要找哪個 team?」
- 「Payment Service 依賴哪些東西?」

核心 entity 類型

Entity	是什麼	例子
Component	軟體單位	python-app (service)、 mobile-sdk (library)、 landing-page (website)
API	對外契約	payment-api (OpenAPI / GraphQL / gRPC schema)
Resource	非程式碼資源	PostgreSQL DB、 S3 bucket、 Kafka topic

Entity	是什麼	例子
System	一群協作的 Component	Checkout System = cart-service + payment-service + inventory-service
Domain	一群相關 System	E-Commerce Domain 涵蓋 Checkout + Catalog + Search
Group	團隊	team-payments
User	個人	alice@company.com

你目前已經在做這個

你的 `python-app/catalog-info.yaml` 就是 Catalog 註冊單:

```

apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: python-app
  description: A simple Python Flask service
  annotations:
    backstage.io/techdocs-ref: dir:.    # ← 連到 TechDocs(第 3 個模組)
spec:
  type: service
  lifecycle: experimental
  owner: christseng89

```

Backstage UI 啟動後,把這個

URL(<https://github.com/christseng89/MasterBackstageIdp/blob/main/python-app/catalog-info.yaml>)註冊進去, 它就會出現在 **Catalog** → **Components** 列表裡。

真正威力(規模大時才看得出來)

當 catalog 裡有 200 個 component、50 個 API、20 個 team:

- **依賴圖**:點 payment-service 看到它依賴 auth-service + inventory-db ,並被 checkout-service 依賴
- **歸屬清楚**:on-call 半夜起來看 catalog 就知道找誰
- **生命週期管理**:過濾「所有 lifecycle: deprecated 的 component」,規劃下架

2. Software Template — 「五分鐘新建一個符合公司標準的服務」

解決的問題

- 「我要新建一個 Python 微服務。怎麼做？」
 - 「先 clone 公司樣板?哪個樣板?」
 - 「CI/CD 怎麼設?抄哪個 repo?」
 - 「Dockerfile 寫對了嗎?」
 - 「要不要先註冊到 Catalog?」
- 答案:全部一個按鈕搞定。

機制

template.yaml 描述「給我這些參數,我幫你生出整套」:

```

apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: python-flask-service
  title: Python Flask Service
  description: Creates a Flask service with Docker, Helm chart, GitOps CI/CD
spec:
  parameters:
    - title: Basic Info
      properties:
        name:
          type: string
          description: Service name (e.g., payment-service)
        owner:
          type: string
          description: Owning team
  steps:
    # 1. 套用 cookiecutter-style template
    - id: fetch
      name: Fetch skeleton
      action: fetch:template
      input:
        url: ./skeleton # 預設骨架(內含 src/, Dockerfile, cicd.yaml...)
        values:
          name: ${ parameters.name }
          owner: ${ parameters.owner }
    # 2. 在 GitHub 開新 repo + push
    - id: publish
      name: Publish to GitHub
      action: publish:github
      input:
        repoUrl: github.com?owner=christseng89&repo=${ parameters.name }
    # 3. 自動註冊到 Catalog
    - id: register
      name: Register in Catalog
      action: catalog:register
      input:
        repoContentsUrl: ${ steps.publish.output.repoContentsUrl }

```

開發者體驗

Backstage UI → **Create** → 選 "Python Flask Service" → 填 name: payment-service owner: team-payments → 按 Create:

- 90 秒後, GitHub 多一個 christseng89/payment-service repo
- 內含 Flask app skeleton、Dockerfile、CI/CD workflow、Helm chart、catalog-info.yaml、mkdocs.yaml

- 新服務自動出現在 Catalog 裡
- CI/CD 已經接好(從你的樣板抄過去)

你還沒有做這個

對你目前的「學習一個 IDP workflow」目標而言,template 是進階主題。等 catalog 跑順了,如果之後想再加類似 microservice,可以把 python-app/ 整個包成 template(目前的 Dockerfile/Helm chart/CI/CD 就是現成的 skeleton)。

3. TechDocs — 「文件跟 code 住在同一個 repo,長同樣的樣」

解決的問題

- 「這個服務的 API 怎麼用？」
- 「Onboarding 文件在 Confluence 還是 Notion?哪一份是最新的？」
- 「文件跟 code 不一致,因為 code 改了 deploy 後沒人記得更新 Confluence」

機制

每個 repo 放兩個東西:

```
python-app/  
├─ mkdocs.yaml      ← 配置文件  
└─ docs/  
    └─ index.md      ← Markdown 內容(可多個檔、可有結構)
```

例如 mkdocs.yaml :

```
site_name: python-app  
nav:  
  - Home: index.md  
  - API: api.md  
  - Deployment: deployment.md
```

Backstage 偵測到 catalog entity 上的 annotation `backstage.io/techdocs-ref: dir:.` 就會:

1. Checkout 你的 repo
2. 把 docs/ 跑過 MkDocs 跟 techdocs-core plugin
3. 產生 HTML
4. 存到後端(可以是 S3、GCS、local disk)
5. 在 component 的 Backstage 頁面側邊塞一個 **Docs** tab

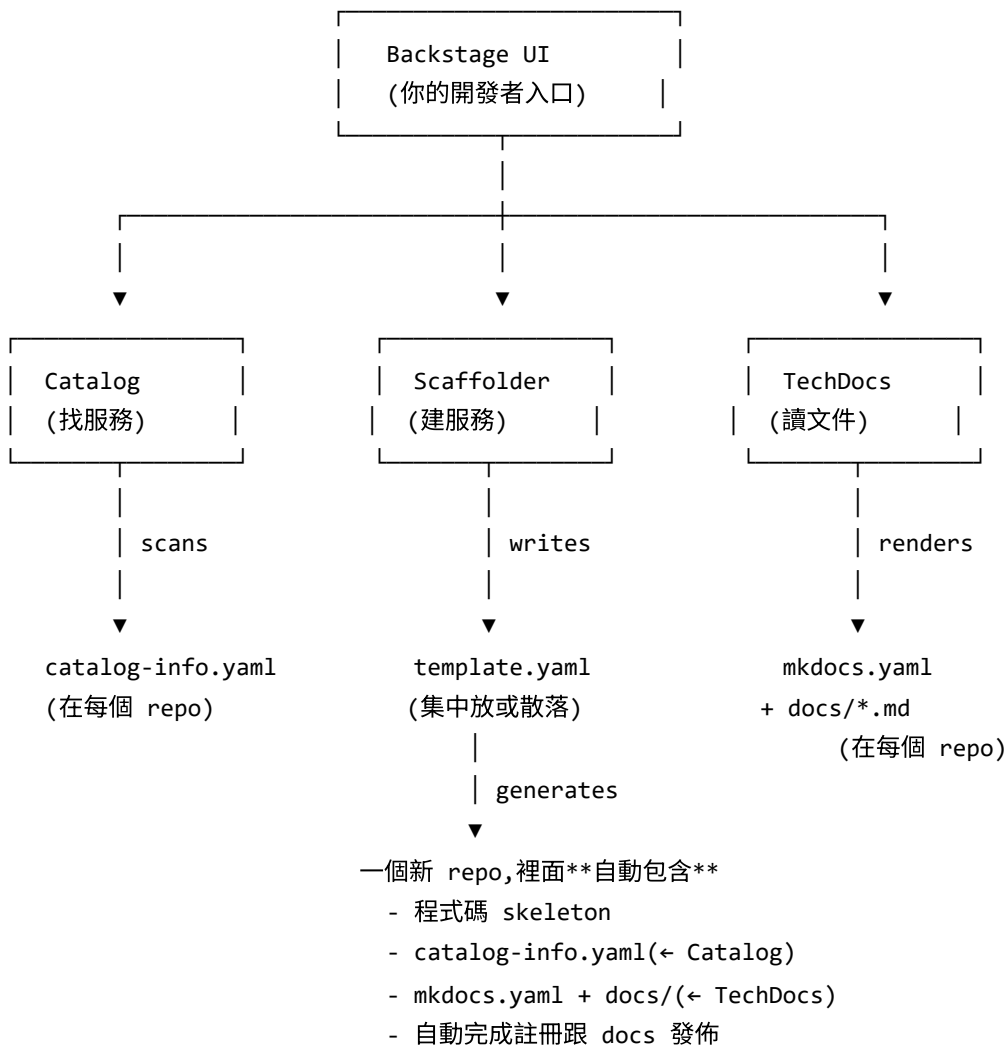
開發者體驗

- 一個 component 的頁面,左側可以同時看到:**Overview / Docs / Dependencies / API / Kubernetes**
- 改文件就是改 markdown,push 之後 Backstage 自動 rebuild(或排程 rebuild)
- 文件版本跟 code 永遠同步(因為它們**就是同一個 commit**)

你目前也已經在做這個

你的 `python-app/mkdocs.yaml` + `python-app/docs/index.md` + `catalog-info.yaml` 裡的
`backstage.io/techdocs-ref: dir:.. annotation = TechDocs` 已經接上了。

三者怎麼合作:把它們連在一起的全景



對你這個 learning project 的建議路徑

階段	你目前已有	還可以做
1. Catalog	✅ python-app/catalog-info.yaml 已寫	在 Backstage UI 註冊這個 catalog URL,確認 python-app 出現在 Catalog
2. TechDocs	✅ mkdocs.yaml + docs/index.md + annotation 都對	寫多一點實際文件(API、deployment runbook),確認 Backstage 的 Docs tab 有渲染出來
3. Template	❌ 還沒做	進階: 把 python-app/ 整個包成 python-microservice-template ,讓 Backstage 一鍵生新 microservice

Catalog + TechDocs 是「**被動**」的(描述既有的東西),Template 是「**主動**」的(生新的東西)。先把前兩個跑通,template 的價值才比較好理解——它生出來的東西就是「自動 catalog 註冊 + 自動 TechDocs 接好」的新 repo。